

Bases de Datos II

Práctico 3

Arquitectura, Escalabilidad y Gestores NoSQL

Nombre: Farias, Gustavo

Comisión: M2025-13

Matrícula: 101662

Repositorio GitHub:

[https://github.com/Lucenear/Tecnicatura_Programacion_UTN/tree/main/3er_Semestre/BBD
D_II](https://github.com/Lucenear/Tecnicatura_Programacion_UTN/tree/main/3er_Semestre/BBD_D_II)

ACTIVIDAD 1: Solución de Infraestructura (Caso: "Ticket-All")

a)

En el caso de Ticket-All, comprar un servidor con triple RAM puede ayudar temporalmente, pero presenta como limitantes que, al tener un único servidor, si se produce una falla, todo el sistema se detiene. También se debe tener en cuenta que al aumentar los recursos del servidor, no necesariamente se aumenta en la misma proporción su rendimiento.

Por el contrario, una escalabilidad horizontal, agregando más servidores al cluster y distribuyendo la carga entre ellos, presenta más ventajas, como por ejemplo, el escalamiento 'ilimitado' según las necesidades y mejor disponibilidad, ya que ante la falla de un nodo se puede distribuir la carga en los demás.

Para Ticket-All, donde hay picos extremos de tráfico (ventas de artistas internacionales), la escalabilidad horizontal es la única solución sostenible. La vertical es una parche temporal que no resuelve el problema raíz: la concentración de carga en un solo punto.

b)

Sugiero implementar sharding basado en hash sobre un campo que garantice distribución uniforme de usuarios/compradores, por ejemplo, ID del usuario, ya que los usuarios son entidades independientes.

Esto porque:

- Si usamos event_id, todos los compradores de un evento popular irían al mismo shard.
- Si usamos timestamp, los nuevos compradores caerían en el último shard.

Un hotspot es un shard (o nodo) que recibe una cantidad desproporcionada de lecturas/escrituras comparado con otros. Esto genera:

- Cuello de botella en ese nodo
- Latencia alta para esos usuarios
- Riesgo de caída por sobrecarga

Para evitarlo, se podría emplear:

- Hashing sobre user_id para evitar que todos los compradores de un evento vayan al mismo shard.
- Alternativa: Sharding compuesto (event_id + user_id), pero sería útil si se trata de eventos frecuentes. En este caso, como el problema es la venta masiva, es mejor priorizar la distribución de usuarios.
- Monitoreo activo: usar herramientas como MongoDB Atlas para detectar shards con alta carga y rebalancear manualmente si es necesario.

c)

En MongoDB, un replica set es un grupo de nodos que mantienen copias idénticas de los datos. Está compuesto por:

- d) Nodo primario: Acepta todas las escrituras y lecturas (por defecto).
- e) Nodos secundarios: Replican datos desde el primario. Pueden realizar lecturas si se configura para realizarlo.

Para la situación en el que el nodo primario falle durante la venta:

- Los secundarios detectan la ausencia del primario.
- Inician un proceso de elección.
- Eligen un nuevo primario entre los secundarios disponibles.
- El nuevo primario comienza a aceptar escrituras.
- Las aplicaciones conectadas a través de mongos redirigen automáticamente las operaciones al nuevo primario.

ACTIVIDAD 2: Modelado y Consistencia (Caso: "Fintech-Plus")

a)

Para la situación de Saldos bancarios y transferencias, aplicaría el modelo ACID, ya que es necesario reflejar consistencia inmediata porque:

- El monto se debite inmediatamente de la cuenta origen.
- Se acredite al mismo tiempo en la cuenta del destinatario .
- La operación no debe quedar 'a medias' si falla algo.

Al contrario, para la situación de el "muro" de noticias financieras donde los usuarios comentan y dan "like", aplicaría el modelo BASE, que brinda consistencia eventual porque:

- El muro siempre está disponible, incluso si algunos servidores fallan.
- Los usuarios pueden comentar y dar like sin esperar a que todo se sincronice.
- Eventualmente, todos vean los mismos likes y comentarios, aunque no sea al instante.

b)

La solución clásica en NoSQL es la desnormalización: guardar los datos del usuario dentro del documento del comentario.

Esto funciona bien porque brinda:

- Lectura ultra rápida: Todo lo necesario está en un solo documento y no hay que buscar en otra colección.
- Se elimina la necesidad, si existe, de realizar joins para obtener los datos necesarios.
- Se generan menos peticiones al backend, lo que brinda menor procesamiento de datos en el servidor, latencia y una mejor experiencia de usuario.

c) Flujo de datos al abrir una app:

- Paso 1: La app se conecta a un mongos

El mongos es como un 'enrutador inteligente'. No almacena datos, solo sabe dónde están. La app no habla directamente con los shards, sino con uno o varios mongos.

- Paso 2: mongos consulta a los Config Servers

Los Config Servers guardan los metadatos del clúster: Cuántos shards hay, qué rangos de datos tiene cada shard, dónde está ubicado cada chunk de la colección comments.

Sin esta información, el mongos no sabría a quién enviar la consulta.

- Paso 3: mongos envía la consulta al shard correcto

Si la shard key es post_id, y el usuario pide comentarios de post_id = 'lalala_123', el mongos busca en los metadatos y descubre que ese post está en el Shard 3 y se envía la consulta solo a ese shard.

- Paso 4: El shard responde y mongos devuelve el resultado

El Shard 3 ejecuta la consulta localmente, devuelve los documentos al mongos, y éste se los entrega a la app.

ACTIVIDAD 3: Investigación

a) Relación entre Teorema CAP y principio BASE

El teorema CAP dice que en un sistema distribuido, sólo se puede tener dos de estas tres cosas:

- Consistencia (todos ven los mismos datos)
- Disponibilidad (siempre puedes leer/escribir)
- Tolerancia a particiones (funciona aunque la red se divida)

No se puede tener las tres al mismo tiempo. Por ejemplo, si la red se parte (partición), y se quiere mantener consistencia, se deben bloquear las escrituras en ambos lados, por lo cual se pierde disponibilidad. Si se quiere mantener disponibilidad, se deben permitir escrituras en ambos lados y se pierde consistencia hasta que se sincronicen.

El principio BASE es la respuesta de los sistemas NoSQL a esto: sacrifican consistencia inmediata para ganar disponibilidad y tolerancia a fallos. Es decir, eligen AP (Disponibilidad + Partición) y aceptan que los datos converjan con el tiempo (consistencia eventual).

Es como decir: "Prefiero que puedas siempre acceder al sistema, aunque a veces veas datos ligeramente desactualizados, antes que bloquearte porque estoy esperando a que todos los nodos se pongan de acuerdo".

Fuente:

<https://www.mongodb.com/docs/manual/core/cap-theorem/>

b) Por qué Redis es más rápido que MongoDB para Leaderboards?

Redis es significativamente más rápido que MongoDB para implementar leaderboards (tablas de clasificación), debido a su arquitectura en memoria y sus estructuras de datos especializadas, diseñadas para mantener el orden de forma automática y constante.

Por qué?

- Todo en memoria: Acceso en nanosegundos. MongoDB lee de disco (más lento).
- Sorted Sets (ZSET): Estructura diseñada específicamente para rankings. Se pueden agregar puntajes y obtener top 10 en milisegundos.
- Comandos atómicos: Como INCR o ZADD, que hacen operaciones seguras y rápidas sin locks.
- Single-threaded: Sin overhead de hilos ni context switching.

MongoDB puede hacerlo, pero requiere índices, agregaciones y lecturas más pesadas. Para un leaderboard en tiempo real (como en juegos o concursos), Redis es imbatible.

Comandos útiles:

LPUSH:

- Función: Inserta uno o varios valores al principio (a la izquierda) de una lista almacenada en una clave (key).

- Comportamiento: Si la clave no existe, se crea una lista vacía antes de realizar la inserción. Si la clave existe pero no contiene una lista, se devuelve un error.
- Uso típico: Implementar pilas (stacks) o colas (FIFO/LIFO), gestionar listas de tareas o historial de eventos.

INCR:

- Función: Incrementa en uno el valor entero almacenado en la clave especificada.
- Comportamiento: Si la clave no existe, se establece en
- antes de realizar la operación. Si el valor no es un entero o es una cadena que no se puede convertir, devuelve un error. Es una operación atómica, por lo que es segura en entornos concurrentes.
- Uso típico: Contadores de visitas, límites de velocidad (rate limiting), generación de IDs secuenciales.

SADD:

- Función: Añade uno o más miembros al conjunto (Set) almacenado en la clave.
- Comportamiento: Si el miembro ya existe en el conjunto, se ignora (garantiza la unicidad). Si la clave no existe, se crea un nuevo conjunto.
- Uso típico: Almacenar etiquetas únicas (tags), gestionar listas de usuarios únicos, categorización, operaciones de conjuntos (uniones, intersecciones).

Fuente:

<https://redis.io/docs/latest/commands/>

c) Por qué Neo4j es mejor que MongoDB para detectar redes de fraude?

Neo4j es una base de datos de grafos nativa. Eso significa que entiende naturalmente las relaciones entre entidades: personas, cuentas, dispositivos, transacciones.

En cambio, MongoDB es documental: guarda objetos JSON, pero las relaciones son secundarias (referencias o arrays embebidos).

Ventaja técnica de Neo4j:

- Las relaciones son ciudadanos de primera clase: tienen dirección, tipo y propiedades.
- Traversals profundos en milisegundos, incluso con millones de nodos.
- Algoritmos integrados: PageRank, Betweenness Centrality, Label Propagation son perfectos para detectar comunidades sospechosas.

Fuente:

<https://neo4j.com/docs/getting-started/current/graphdb-concepts/>